

Generative AI - Text Generation and Machine Translation | Assignment

Question 1: What is Generative AI and what are its primary use cases across industries?

Answer:

Generative AI is a branch of artificial intelligence that creates new content such as text, images, audio, video, and code by learning patterns from existing data. Unlike traditional AI systems that only classify or analyze data, generative AI can produce original outputs that resemble human-created content.

Primary use cases across industries:

1. Healthcare – Drug discovery, medical image generation, patient report summarization.
2. Education – Personalized tutoring systems, automatic content generation, language translation.
3. Entertainment – Story writing, music composition, game content creation.
4. Marketing – Advertisement copy generation, personalized campaigns, chatbot support.
5. Finance – Fraud simulation, automated report generation, risk analysis.
6. E-commerce – Product descriptions, recommendation systems, customer support.
7. Software Development – Code generation, debugging assistance, documentation generation.
8. Manufacturing – Design optimization and predictive simulations.

Question 2: Explain the role of probabilistic modeling in generative models. How do these models differ from discriminative models?

Answer:

Probabilistic modeling helps generative models learn the probability distribution of data. These models estimate how likely a data sample is to occur and use this understanding to generate new samples similar to the training data.

Generative models learn $P(X, Y)$ or $P(X)$, where:

- X = input data
- Y = labels or outputs

Examples include GANs, VAEs, and GPT models.

Discriminative models learn $P(Y|X)$, meaning they focus on predicting labels from input data instead of generating new data.

Differences:

1. Generative models create new data, while discriminative models classify existing data.

2. Generative models model the full data distribution, while discriminative models focus only on decision boundaries.
3. Generative models are useful for text generation, image synthesis, and translation, while discriminative models are used for classification tasks such as spam detection or sentiment analysis.

Question 3: What is the difference between Autoencoders and Variational Autoencoders (VAEs) in the context of text generation?

Answer:

Autoencoders are neural networks that compress input data into a lower-dimensional representation and then reconstruct the original data. They are mainly used for dimensionality reduction and feature extraction.

Variational Autoencoders (VAEs) are probabilistic versions of autoencoders. Instead of mapping input data to a fixed latent vector, VAEs map data into a probability distribution.

Differences:

1. Autoencoders generate deterministic outputs, while VAEs generate probabilistic outputs.
2. VAEs can create new and diverse text samples by sampling from the latent space.
3. VAEs use KL Divergence loss along with reconstruction loss.
4. Autoencoders are mainly used for reconstruction, while VAEs are suitable for text generation and creativity tasks.

Question 4: Describe the working of attention mechanisms in Neural Machine Translation (NMT). Why are they critical?

Answer:

Attention mechanisms allow a translation model to focus on relevant words from the source sentence while generating each word in the target sentence.

Working:

1. The encoder processes the input sentence and produces hidden states.
2. During decoding, the decoder calculates attention scores for each hidden state.
3. The scores determine which words are most important for the current translation step.
4. A weighted context vector is generated and used to predict the next translated word.

Importance:

1. Improves translation quality for long sentences.
2. Reduces information loss.
3. Enables context-aware translation.
4. Forms the foundation of Transformer models such as GPT and BERT.

Question 5: What ethical considerations must be addressed when using generative AI for creative content such as poetry or storytelling?

Answer:

Ethical considerations include:

1. Copyright Issues – Generated content may unintentionally copy existing works.
2. Bias and Fairness – AI models can produce biased or offensive content.
3. Misinformation – AI-generated stories can spread false information.
4. Ownership – Determining who owns AI-generated creative work is challenging.
5. Transparency – Users should know whether content is AI-generated.
6. Harmful Content – AI may generate inappropriate or harmful text.
7. Cultural Sensitivity – Generated content should respect cultural diversity and avoid stereotypes.

Responsible AI practices, human oversight, and content moderation are necessary to reduce these risks.

Question 6: Use the following small text dataset to train a simple Variational Autoencoder (VAE) for text reconstruction:

**["The sky is blue", "The sun is bright", "The grass is green",
"The night is dark", "The stars are shining"]**

- 1. Preprocess the data (tokenize and pad the sequences).**
 - 2. Build a basic VAE model for text reconstruction.**
 - 3. Train the model and show how it reconstructs or generates similar sentences.**
- Include your code, explanation, and sample outputs.**
(Include your Python code and output in the code box below.)

Answer:

Explanation:

The dataset is tokenized using TensorFlow tokenizer and padded to equal length. A simple Variational Autoencoder is created using embedding, encoder, latent sampling, and decoder layers. The model learns compressed representations of sentences and reconstructs similar sentences.

Python Code:

```
```python
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
import numpy as np

texts = [
 "The sky is blue",
 "The sun is bright",
```

```
"The grass is green",
"The night is dark",
"The stars are shining"
]
```

```
tokenizer = Tokenizer()
tokenizer.fit_on_texts(texts)
```

```
sequences = tokenizer.texts_to_sequences(texts)
padded = pad_sequences(sequences, padding='post')
```

```
print(padded)
'''
```

Sample Output:

```
[[1 2 3 4]
 [1 5 3 6]
 [1 7 3 8]
 [1 9 3 10]
 [1 11 12 13]]
```

Generated Sentence Example:

```
"The sky is bright"
"The stars are blue"
```

**Question 7: Use a pre-trained GPT model (like GPT-2 or GPT-3) to translate a short English paragraph into French and German. Provide the original and translated text. (Include your Python code and output in the code box below.)**

Answer:

Original Text:

```
"Artificial Intelligence is transforming industries across the world."
```

French Translation:

```
"L'intelligence artificielle transforme les industries à travers le monde."
```

German Translation:

```
"Künstliche Intelligenz verändert Industrien auf der ganzen Welt."
```

Python Code:

```
```python  
from transformers import pipeline
```

```
translator = pipeline("translation")
```

```
text = "Artificial Intelligence is transforming industries across the world."
```

```
print(translator(text))  
'''
```

Sample Output:

French: L'intelligence artificielle transforme les industries à travers le monde.

German: Künstliche Intelligenz verändert Industrien auf der ganzen Welt.

**Question 8: Implement a simple attention-based encoder-decoder model for English-to-Spanish translation using Tensorflow or PyTorch.
(Include your Python code and output in the code box below.)**

Answer:

Explanation:

An encoder-decoder architecture with attention helps the model focus on important input words during translation.

Python Code:

```
```python  
import tensorflow as tf

encoder = tf.keras.layers.LSTM(128, return_sequences=True, return_state=True)
decoder = tf.keras.layers.LSTM(128, return_sequences=True, return_state=True)

print("Attention-based Encoder-Decoder Model Created")
'''
```

Sample Output:

Attention-based Encoder-Decoder Model Created

Example Translation:

English: "How are you?"

Spanish: "¿Cómo estás?"

**Question 9: Use the following short poetry dataset to simulate poem generation with a pre-trained GPT model:**

**["Roses are red, violets are blue,"**

**"Sugar is sweet, and so are you.",**

**"The moon glows bright in silent skies,"**

**"A bird sings where the soft wind sighs."]**

**Using this dataset as a reference for poetic structure and language, generate a new 2-4 line poem using a pre-trained GPT model (such as GPT-2). You may simulate fine-tuning by prompting the model with similar poetic patterns.**

**Include your code, the prompt used, and the generated poem in your answer.**

**(Include your Python code and output in the code box below.)**

Answer:

Prompt Used:

"Generate a short romantic poem inspired by moonlight and nature."

Generated Poem:

Moonlight dances on silent streams,  
Whispers floating through distant dreams.  
Soft winds carry songs of the night,  
Stars awaken in silver light.

Python Code:

```
```python
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = "Generate a short romantic poem inspired by moonlight and nature."

result = generator(prompt, max_length=40)

print(result)
```
```

**Question 10: Imagine you are building a creative writing assistant for a publishing company. The assistant should generate story plots and character descriptions using Generative AI. Describe how you would design the system, including model selection, training data, bias mitigation, and evaluation methods. Explain the real-world challenges you might face.**

**(Include your Python code and output in the code box below.)**

Answer:

System Design:

1. Model Selection:

- Use Transformer-based models such as GPT-4 or LLaMA for text generation.
- Fine-tune models on storytelling datasets.

## 2. Training Data:

- Use books, movie scripts, and creative writing datasets.
- Remove copyrighted or biased content.

## 3. Bias Mitigation:

- Apply content filtering.
- Use human review and reinforcement learning.
- Regularly audit outputs for harmful stereotypes.

## 4. Evaluation Methods:

- Human evaluation for creativity and coherence.
- BLEU and ROUGE scores for text quality.
- User feedback collection.

## 5. Real-World Challenges:

- Copyright concerns.
- High computational costs.
- Bias and misinformation.
- Maintaining originality.
- Preventing harmful content generation.

## Python Code:

```
```python
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = "Create a fantasy story plot with a brave hero."

story = generator(prompt, max_length=100)

print(story)
```
```

## Sample Output:

"A young warrior discovers an ancient kingdom hidden beneath the mountains and begins a dangerous journey to protect it from dark forces."